

90 Days SDET Interview Prep

Day 5

1. What is Exception Handling?

- Exception handling is a mechanism **in Java to manage unexpected runtime failures** in a controlled way so that the *normal flow of the program is not interrupted.*
- **In Automation Frameworks:**
 - Instead of a random crash, exception handling allows us to fail tests with proper logs, screenshots, and meaningful error messages.
 - Failures such as `NoSuchElementException`, `TimeoutException`, or null API responses are expected during the failure.

In Java Exceptions are handled using

1. Try catch
2. Throws keyword

Best way to Handle Exception will always be Try Catch!

Code Example:

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Handled exception: " + e.getMessage());  
}
```

Common Exceptions in Selenium and Rest Assured that you should Remember:

Selenium WebDriver:

1. **NoSuchElementException**: Selenium is **unable to find the element in the DOM**.
2. **TimeoutException**: Page or element loaded slower than expected
3. **StaleElementReferenceException**: The element reference is **no longer valid** because the DOM has changed.
4. **ElementNotInteractableException**: Element exists in DOM but cannot be interacted with.
5. **ElementClickInterceptedException**: Another element is **blocking the click action**.
6. **InvalidSelectorException**: The locator syntax is invalid
7. **NoSuchFrameException**: Trying to switch to a frame that does not exist.
8. **NoSuchWindowException**: Trying to switch to a window or tab that is already closed or not available.
9. **WebDriverException**: when WebDriver cannot perform an operation.

Rest Assured:

1. **AssertionError**: The actual API response does not match the expected condition.
2. **IllegalArgumentException**: Wrong header or parameter value
3. **JsonPathException**: Wrong JSON structure assumption
4. **ConnectException**: Server is down
5. **SocketTimeoutException**: Slow API response
6. **SSLHandshakeException**: Invalid or expired SSL certificate

2. Checked vs. Unchecked Exceptions

Feature	Checked Exception	Unchecked (Runtime) Exception
Check Time	Compile time.	Runtime [happens during execution]
Handling	Mandatory (try-catch or throws)	Optional
Cause	External factors (File, DB, Network,)	Programming logic mistakes
Example	IOException, FileNotFoundException	NullPointerException, ArrayIndexOutOfBounds Selenium Exceptions RestAssuredExceptions

Remember: Many frameworks wrap Checked exceptions into Unchecked ones to keep Page Object methods clean and avoid cluttering method signatures with throws declarations.

```
public void clickOn(WebElement element) {
    try {
        logger.info("Element Found and now performing Click");
        element.click();
    } catch (Exception e) {
        logger.error("Failed to click on element: " + element);
        throw new RuntimeException("Click operation failed", e);
    }
}
```

3. Difference Between throw and throws

- **Throw:** Used inside a method to explicitly trigger an exception when a specific condition occurs.
 - **Example:** `throw new RuntimeException("Username missing in test data");`

```
public class LoginPage {

    public void enterUsername(String username) {
        // throw is used INSIDE the method to explicitly throw an exception
        if (username == null || username.isEmpty()) {
            throw new RuntimeException("Username missing in test data");
        }

        System.out.println("Username entered: " + username);
    }

    public static void main(String[] args) {
        LoginPage page = new LoginPage();
        page.enterUsername(null); // This will throw RuntimeException
    }
}

**Output:**
Exception in thread "main" java.lang.RuntimeException: Username missing in test data
```

- **throws (The Warning):** Used in the method signature to declare that a method *might* pass an exception to the caller.
 - **Example:** `public void readFile() throws IOException`
`{ ... }`

```
import java.io.FileInputStream;
import java.io.IOException;

public class FileHandler {

    // throws is used in METHOD SIGNATURE to declare exception
    public void readFile() throws IOException {
        FileInputStream fis = new FileInputStream("test.txt");
        System.out.println("File opened successfully");
    }

    public static void main(String[] args) {
        FileHandler handler = new FileHandler();

        try {
            handler.readFile();
        } catch (IOException e) {
            System.out.println("Caught exception: " + e.getMessage());
        }
    }
}

/**Output (if test.txt doesn't exist):**
Caught exception: test.txt (No such file or directory)
```

4. Custom Exceptions in Automation

Custom exceptions are user-defined classes created to represent application-specific failures.

Why we need them:

Generic exceptions (like `NullPointerException`) don't provide enough context in a Jenkins or CI report.

Custom exceptions make failures readable, meaningful, and easier to debug.

Implementation Example:

```
// 1. Create the Custom Exception Class
public class InvalidTestDataException extends RuntimeException {
    public InvalidTestDataException(String message) {
        super(message);
    }
}

// 2. Use it in your Framework
if (testData.isEmpty()) {
    throw new InvalidTestDataException("Login test data is missing from the
Excel/JSON file");
}
```

5. Purpose of Finally Block?

Finally Block is executed regardless of whether an exception was thrown or caught.

- Scenario: Closing database connections or quitting the WebDriver instance.
- Rule: Even if the test crashes, the finally block ensures the browser closes, preventing memory leaks on your execution grid.

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class WebDriverTest {

    public static void main(String[] args) {
        WebDriver driver = null;

        try {
            driver = new ChromeDriver();
            driver.get("https://www.google.com");

            // Simulate a test failure
            throw new RuntimeException("Test failed unexpectedly");
        } catch (Exception e) {
            System.out.println("Exception caught: " + e.getMessage());
        } finally {
            // This ALWAYS executes browser will close even if test fails
            if (driver != null) {
                driver.quit();
                System.out.println("Browser closed successfully");
            }
        }
    }
}
```